



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 09

NOMBRE COMPLETO: Camacho Ignacio Violeta

N° de Cuenta: 319061345

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 23-04-2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

1. Hacer que su carro recorra la pista Adecuadamente (curvas, inclinación en rampa, faro frontal ilumina la pista) desde el inicio hasta el punto extremo y ahí se detenga. Se puede repetir la animación si se presiona una tecla designada.





```

429 struct Waypoint {
430     glm::vec3 posicion;
431     float rotY;    // giro lado a lado (curvas)
432     float rotZ;    // inclinación (rampa)
433 };
434
435 std::vector<Waypoint> ruta = {
436     { glm::vec3(2.0f, -1.0f, -4.0f), 0.0f, 0.0f }, // inicio
437     { glm::vec3(25.0f, -1.0f, -4.0f), 5.0f, 0.0f },
438     { glm::vec3(45.0f, -1.0f, -10.0f), 25.0f, 0.0f },
439     { glm::vec3(56.0f, 0.5f, -11.0f), -30.0f, 7.0f },
440     { glm::vec3(67.0f, 2.7f, 0.0f), -50.0f, 10.0f },
441     { glm::vec3(75.0f, 4.5f, 6.5f), -20.0f, 17.0f },
442     { glm::vec3(85.0f, 8.0f, 7.0f), 15.0f, 20.0f },
443     { glm::vec3(103.0f, 16.5f, 2.5f), 7.0f, 25.0f },
444     { glm::vec3(120.0f, 23.5f, 3.0f), -10.0f, 15.0f },
445     { glm::vec3(140.0f, 26.5f, 8.0f), -18.0f, 6.0f },
446     { glm::vec3(160.0f, 27.0f, 14.0f), -18.0f, -1.0f },
447     { glm::vec3(177.0f, 25.0f, 20.0f), -20.0f, -5.0f }, // fin
448 };
449
450 int waypointActual = 0;
451 float tRuta = 0.0f;    // 0.0 a 1.0 entre waypoint actual y el siguiente
452 float velocidadRuta = 0.02f; // entre puntos
453 bool carroActivo = false; // se activa con la tecla P

```

```

456 while (!mainwindow.getShouldClose())
457 {
458     GLfloat now = glfwGetTime();
459     deltaTime = now - lastTime;
460     deltaTime += (now - lastTime) / limitFPS;
461     lastTime = now;
462
463     anguloVaria += 0.5f * deltaTime;
464
465     if (carroActivo && waypointActual < (int)ruta.size() - 1)
466     {
467         tRuta += velocidadRuta * deltaTime;
468
469         if (tRuta >= 1.0f)
470         {
471             tRuta = 0.0f;
472             waypointActual++;
473
474             // llegó al final, se detiene
475             if (waypointActual >= (int)ruta.size() - 1)
476             {
477                 waypointActual = (int)ruta.size() - 1;
478                 carroActivo = false;
479             }
480         }
481     }

```

```

483 // Interpolación entre waypoint actual y el siguiente
484 int siguiente = glm::min(waypointActual + 1, (int)ruta.size() - 1);
485 glm::vec3 posCarro = glm::mix(ruta[waypointActual].posicion, ruta[siguiente].posicion, tRuta);
486 float rotYCarro = glm::mix(ruta[waypointActual].rotY, ruta[siguiente].rotY, tRuta);
487 float rotZCarro = glm::mix(ruta[waypointActual].rotZ, ruta[siguiente].rotZ, tRuta);
488
489 // Llantas giran solo si el carro se mueve
490 if (carroActivo) rotLlanta += rotLlantaOffset * deltaTime;

```



```

497 // Tecla P: reinicia y activa la animación
498 if (mainWindow.getAnimacionCoche())
499 {
500     waypointActual = 0;
501     tRuta = 0.0f;
502     rotllanta = 0.0f;
503     carroActivo = true;
504     mainWindow.resetAnimacionCoche(); // no reiniciar cada frame
505 }

569 // ===== cuerpo carro =====
570 model = glm::mat4(1.0);
571 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); // corrección base del modelo hacia -x
572 model = glm::translate(model, posCarro);
573 model = glm::rotate(model, rotYCarro * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); // curvas lado a lado
574 model = glm::rotate(model, rotZCarro * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); // inclinación arriba abajo
575 modelaux = model; // heredar todo lo anterior
576 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
577 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
578 Carro_cuerpo.RenderModel();
579
580 // ===== FARO (heredar dirección del carro) =====
581 glm::mat4 modelFaro = modelaux;
582 modelFaro = glm::translate(modelFaro, glm::vec3(5.5f, 1.5f, 0.0f));
583 glm::vec3 posFaro = glm::vec3(modelFaro[3]);
584 glm::vec3 dirFaro = glm::normalize(glm::vec3(modelaux[0])); // eje X local del carro
585 spotLights[2].SetFlash(posFaro, dirFaro);
586
587 // ===== cofre =====
588 model = modelaux;
589 model = glm::translate(model, glm::vec3(2.0f, 3.0f, 0.0f));
590 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
591 Carro_cofre.RenderModel();
592
593 // ===== llantal delantera izquierda =====
594 model = modelaux;
595 model = glm::translate(model, glm::vec3(4.5f, 1.0f, -3.0f));
596 model = glm::rotate(model, -rotllanta * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
597 color = glm::vec3(0.5f, 0.5f, 0.5f);
598 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
599 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
600 Carro_llantal.RenderModel();

```

Para lograr que el carro siguiera la trayectoria de la pista correctamente se implementó un sistema de waypoints, cada waypoint es un punto de control definido por la posición en el espacio (X, Y, Z) que son los primeros 3 parametros, el ángulo de rotación horizontal rotY para representar los giros de lado a lado, y el ángulo de inclinación rotZ para representar la pendiente de la rampa.

Una vez definidos los waypoints, en cada frame del game loop se usa la función glm::mix para interpolar suavemente entre el waypoint actual y el siguiente, usando una variable tRuta que va de 0.0 a 1.0, cuando tRuta llega a 1.0, se avanza al siguiente waypoint y tRuta se reinicia a 0.0, esto produce un movimiento fluido y continuo a lo largo de toda la ruta sin saltos bruscos.

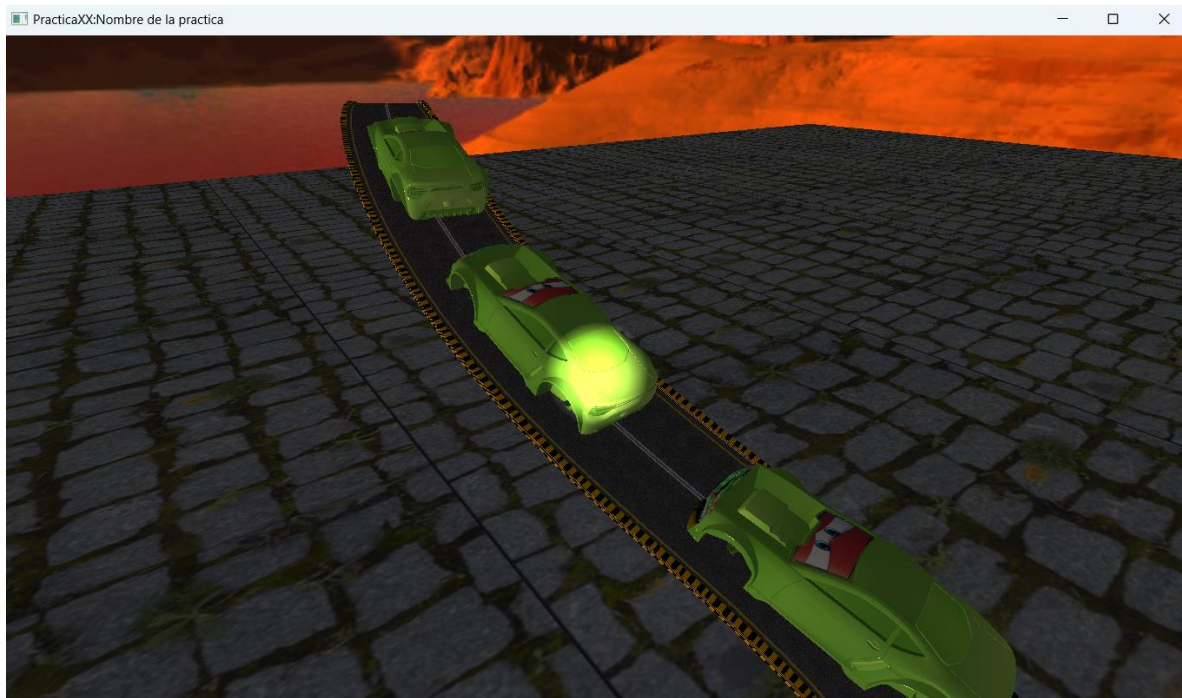
Al modelo del coche primero se le aplicó una rotación de 180° como corrección base del modelo (ya que el carro originalmente apunta hacia el eje -X), luego la traslación a la posición interpolada, después la rotación en Y para las curvas, y finalmente la rotación en Z para la inclinación y se captura en la matriz para heredar por jerarquía la misma posición y orientación.

El faro frontal del carro se implementó como un SpotLight cuya posición y dirección se actualizan cada frame extrayendo el eje X local de la matriz, esto

garantiza que el haz de luz siempre apunte hacia donde el carro está mirando sin importar las curvas o la inclinación, la animación se activa presionando la tecla P, implementada desde la clase Window que se consume con un método resetAnimacionCoche para evitar que se active repetidamente.

La técnica de los waypoints consiste en definir manualmente una lista de estos puntos a lo largo de la pista, para obtenerlos se fueron aplicando transformaciones al modelo del coche para obtener las posiciones y el angulo que se le debía aplicar y en que posición a cada punto para dar el giro y la inclinación en la posición correcta en el espacio e ir creando una especie de frames que sigue el movimiento del coche

```
643 // ===== cuerpo carro =====
644 model = glm::mat4(1.0);
645 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //corrección pq originalmente apunta hacia -x
646
647 //model = glm::translate(model, glm::vec3(2.0f, -1.0f, -4.0f)); //posicion inicial
648
649 /*
650 model = glm::translate(model, glm::vec3(25.0f, -1.0f, -4.0f)); //posicion
651 model = glm::rotate(model, 5 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //giro lado a lado
652 */
653
654 /*
655 model = glm::translate(model, glm::vec3(45.0f, -1.0f, -10.0f)); //posicion
656 model = glm::rotate(model, 25 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //giro lado a lado
657 */
658
659 /*
660 model = glm::translate(model, glm::vec3(56.0f, 0.0f, -11.0f)); //posicion
661 model = glm::rotate(model, -30 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //giro lado a lado
662 model = glm::rotate(model, 7 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //giro hacia arriba y hacia abajo
663 */
664
665 /*
666 model = glm::translate(model, glm::vec3(67.0f, 2.0f, 0.0f)); //posicion
667 model = glm::rotate(model, -50 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); //giro lado a lado
668 model = glm::rotate(model, 10 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f)); //giro hacia arriba y hacia abajo
669 */
701 */
702
703 /*
704 model = glm::translate(model, glm::vec3(160.0f, 26.5f, 14.0f)); //posicion
705 model = glm::rotate(model, -18 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
706 model = glm::rotate(model, -1 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
707 */
708
709 /*
710 model = glm::translate(model, glm::vec3(177.0f, 25.0f, 20.0f)); //posicion final
711 model = glm::rotate(model, -20 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
712 model = glm::rotate(model, -5 * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
713 */
714
715 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
716 //Carro_cuerpo.RenderModel();
717
718
```



2.-Separar de la Nave el otra ala y las dos hélices, Hacer que sobrevuele la pista adecuadamente(jerarquía, aleteo, giro de hélices, faro ilumina la pista) pero en sentido contrario, inicia en el punto extremo y aterriza junto al punto inicial. Esta animación no se puede repetir.





```

398 //----- faro nave -----
399 spotLights[1] = SpotLight(
400     1.0f, 1.0f, 0.0f,    // color amarillo
401     1.0f, 5.0f,         // intensidad ambiental y difusa
402     0.0f, 0.0f, 0.0f,    // posición
403     0.0f, -5.0f, 0.0f,   // apunta hacia el piso -y
404     0.4f, 0.4f, 0.4f,    // atenuación
405     25.0f                // angulo
406 );
407 spotLightCount++;
408 //-----

```

```

467 // Waypoints de la nave: sentido invertido, separar de la pista para q flote
468 std::vector<Waypoint> rutaNave = {
469     { glm::vec3(177.0f, 28.0f, 20.0f), -20.0f, -5.0f }, // inicio
470     { glm::vec3(160.0f, 30.0f, 14.0f), -18.0f, -1.0f },
471     { glm::vec3(140.0f, 29.5f, 8.0f), -18.0f, 6.0f },
472     { glm::vec3(120.0f, 26.5f, 3.0f), -10.0f, 15.0f },
473     { glm::vec3(103.0f, 19.5f, 2.5f), 7.0f, 25.0f },
474     { glm::vec3(85.0f, 11.0f, 7.0f), 15.0f, 20.0f },
475     { glm::vec3(75.0f, 7.5f, 6.5f), -20.0f, 17.0f },
476     { glm::vec3(67.0f, 5.7f, 0.0f), -50.0f, 10.0f },
477     { glm::vec3(56.0f, 3.5f, -11.0f), -30.0f, 7.0f },
478     { glm::vec3(45.0f, 2.0f, -10.0f), 25.0f, 0.0f },
479     { glm::vec3(25.0f, 2.0f, -4.0f), 5.0f, 0.0f },
480     { glm::vec3(2.0f, 2.0f, -4.0f), 0.0f, 0.0f }, // aterriza
481 };

```

```

489 // variables nave
490 int waypointNaveActual = 0;
491 float tNave = 0.0f;
492 float velocidadNave = 0.015f;
493 bool naveActiva = true; // arranca automáticamente, no se puede repetir
494 bool naveTerminada = false;
495

```



```

526 // animación nave
527 if (naveActiva && !naveTerminada && waypointNaveActual < (int)rutaNave.size() - 1)
528 {
529     tNave += velocidadNave * deltaTime;
530     if (tNave >= 1.0f)
531     {
532         tNave = 0.0f;
533         waypointNaveActual++;
534         if (waypointNaveActual >= (int)rutaNave.size() - 1)
535         {
536             waypointNaveActual = (int)rutaNave.size() - 1;
537             naveTerminada = true; // se detiene y no se repite
538         }
539     }
540 }

690 //Aquí va la nave con jerarquía de modelos, completar
691 // ===== nave =====
692 model = glm::mat4(1.0f);
693 model = glm::rotate(model, 180 * toRadians, glm::vec3(0.0f, 1.0f, 0.0f)); // misma corrección que el carro
694 model = glm::translate(model, posNave);
695 model = glm::rotate(model, rotYNave * toRadians, glm::vec3(0.0f, 1.0f, 0.0f));
696 glm::mat4 modelNave = model;
697
698 model = glm::scale(model, glm::vec3(3.0f, 3.0f, 3.0f));
699 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
700 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
701 Nave_M.RenderModel();
702
703 // ===== ala =====
704 model = modelNave;
705 model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.5f));
706 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
707 Ala_M.RenderModel();
708
709 // ===== faro nave (apuntar al piso seguir nave) =====
710 glm::mat4 modelFaroNave = modelNave;
711 modelFaroNave = glm::translate(modelFaroNave, glm::vec3(0.0f, -1.0f, 0.0f)); // debajo de la nave
712 glm::vec3 posFaroNave = glm::vec3(modelFaroNave[3]);
713 glm::vec3 dirFaroNave = glm::vec3(0.0f, -1.0f, 0.0f); // q siempre apunte al piso
714 spotLights[1].SetFlash(posFaroNave, dirFaroNave);

```

La solución de la nave utiliza exactamente la misma estructura de waypoints del carro, con la diferencia de que el orden de los puntos está invertido de fin a inicio y los valores de Y en cada punto están ligeramente elevados para que la nave vuele por encima de la pista en lugar de chocar con ella, las coordenadas X, Z y las rotaciones son las mismas que en la ruta del carro, lo que garantiza que la nave siga el mismo camino geométrico pero en dirección contraria.

Para que la nave siguiera la misma interpretación de coordenadas que el carro, se aplicó la misma rotación de corrección de 180° antes del translate, sin esta corrección, la nave interpretaba el eje X al revés y se desplazaba en la dirección opuesta a la esperada generando un efecto en el movimiento de tipo espejo.

El faro de la nave apunta permanentemente hacia el piso mediante un vector de dirección fijo (0, -1, 0), para que la posición del faro fuera correcta en coordenadas de mundo, se calculó tomando en cuenta la corrección de 180° que invierte los ejes X y Z, por lo que la posición real se obtiene negando los componentes X y Z de posNave.

Al ser una animación no repetible, se usa un booleano naveTerminada que se activa cuando el waypointNaveActual alcanza el último punto, una vez activado, la

condición de movimiento deja de cumplirse y la nave permanece estática en su posición final sin ninguna tecla que permita reiniciarla.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

Al principio en el segundo ejercicio la nave se movía en la dirección contraria a la esperada aunque los waypoints eran los mismos que el carro, la causa fue que al carro se le aplica una rotación de 180° antes del translate, lo que invertía los ejes, al agregar la misma corrección a la nave ambos modelos interpretaron las coordenadas de manera idéntica.

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

Los ejercicios de esta práctica presentaron una complejidad alta ya que el concepto de waypoints es relativamente sencillo de entender pero su correcta implementación depende entender el orden de las operaciones en la cadena de matrices no es arbitrario, además de tener que calcular manualmente una por una las posiciones y direcciones del modelo con respecto a la pista.

b. Comentarios generales:

Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

La práctica estuvo bien estructurada en cuanto a la progresión de dificultad solamente hubo problemas para su conclusión debido a la carga de trabajo de otras materias, ya que si bien se implementan conceptos que se implementaron en el ejercicio de clase el tener que implementar varios movimientos al mismo tiempo resulta en un alto consumo de tiempo para ajustar y calcular manualmente sus posiciones, giros, etc.

c. Conclusión

Esta práctica permitió comprender de manera práctica el sistema de waypoints como mecanismo para guiar objetos a lo largo de trayectorias complejas a diferencia de la animación simple por banderas y condicionales vista anteriormente, esta técnica permite representar rutas

con curvas, cambios de elevación y orientación variable sin necesidad de definir un algoritmo matemático explícito para cada tramo, sino simplemente especificando los estados clave del objeto y dejando que la interpolación construya el movimiento intermedio.

Bibliografía en formato APA

- Caini, B. (2023). Learn OpenGL: Graphics programming. learnopengl.com. Recuperado de <https://learnopengl.com>
- G-Truc Creation. (2023). GLM: OpenGL Mathematics (Version 0.9.9). Recuperado de <https://glm.g-truc.net/0.9.9/index.html>
- Leal, D. (2021). OpenGL tutorial: Transformaciones y matrices. OpenGL-tutorial.org. Recuperado de <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-3-matrices/>
- The Khronos Group. (2023). OpenGL reference pages. Recuperado de <https://registry.khronos.org/OpenGL-Refpages/gl4/>
- Tecnologías Interactivas y Computación Gráfica. (22 oct 2024). Animación por keyframes en OpenGL: movimiento y rotación suaves [Tutorial completo] [Video]. YouTube. Recuperado de <https://www.youtube.com/watch?v=5l1A6Lxa3GQ>